

Lab Instructions

In this lab, we study **multiprocessing for CPU-bound AI preprocessing**, using a serial baseline, ProcessPoolExecutor, chunk-level map-reduce, benchmarking, and correctness checks. Python's multiprocessing module supports process-based parallelism and can side-step the Global Interpreter Lock by using subprocesses, which allows CPU-bound Python work to use multiple cores. Python's concurrent.futures.ProcessPoolExecutor provides a higher-level interface for running callables in separate processes, and its map() supports chunksize, which can significantly affect performance for long iterables.

[\[mpitutorial.com\]](#) [\[mpitutorial.com\]](#)

Learning Objectives

By the end, students should be able to:

- Explain why multiprocessing helps CPU-bound Python code.
- Use ProcessPoolExecutor.
- Write top-level worker functions that can be used by subprocesses.
- Use the if __name__ == "__main__" guard.
- Measure runtime, speedup, efficiency, and throughput.
- Understand process overhead, pickling overhead, and chunk-size tradeoffs.
- Implement a simple map-reduce workflow.

Full Example Code: July23_multiprocessing_lab.py

Student Results Table

Students should produce a table like this:

Method	Workers	Chunk Size	Mean Time	Speedup	Efficiency	Items/sec	Correct
serial	1	0		1.00	1.00		True
item-level	2	1					

Method	Workers	Chunk Size	Mean Time	Speedup	Efficiency	Items/sec	Correct
item-level	4	100					
map-reduce	2	500					
map-reduce	4	1000					
map-reduce	8	5000					

Discussion Questions

Ask students:

1. Why can multiprocessing speed up CPU-bound Python code?
 2. Why might workers=8 not be twice as fast as workers=4?
 3. Why can chunksize=1 be very slow?
 4. Why might a very large chunk size also be bad?
 5. What data is being sent from the main process to workers?
 6. What data comes back from workers?
 7. Is your parallel code correct compared with the serial version?
 8. How does map-reduce reduce communication overhead?
-

Parallel Word Count

This is good for students interested in NLP, text processing, or LLM preprocessing.

```
from collections import Counter
```

```
from concurrent.futures import ProcessPoolExecutor
```

```
import time
```

```
def make_fake_lines(n=100_000):
```

```
    words = [
```

```
"ai", "model", "data", "parallel", "python",  
"thread", "process", "training", "inference",  
]
```

```
lines = []
```

```
for i in range(n):
```

```
    line = " ".join(words[(i + j) % len(words)] for j in range(20))
```

```
    lines.append(line)
```

```
return lines
```

```
def chunk_list(items, chunk_size):
```

```
    for i in range(0, len(items), chunk_size):
```

```
        yield items[i:i + chunk_size]
```

```
def count_words(lines):
```

```
    counter = Counter()
```

```
    for line in lines:
```

```
        tokens = line.lower().split()
```

```
        counter.update(tokens)
```

```
    return counter
```

```
def merge_counters(counters):
```

```
total = Counter()
```

```
for c in counters:
```

```
    total.update(c)
```

```
return total
```

```
def serial_word_count(lines):
```

```
    return count_words(lines)
```

```
def parallel_word_count(lines, workers=4, chunk_size=5000):
```

```
    chunks = list(chunk_list(lines, chunk_size))
```

```
    with ProcessPoolExecutor(max_workers=workers) as executor:
```

```
        partials = list(executor.map(count_words, chunks))
```

```
    return merge_counters(partials)
```

```
def main():
```

```
    lines = make_fake_lines()
```

```
    start = time.perf_counter()
```

```
    serial = serial_word_count(lines)
```

```
    serial_time = time.perf_counter() - start
```

```
    start = time.perf_counter()
```

```

parallel = parallel_word_count(lines, workers=4, chunk_size=5000)

parallel_time = time.perf_counter() - start

print("serial time:", serial_time)
print("parallel time:", parallel_time)
print("speedup:", serial_time / parallel_time)
print("correct:", serial == parallel)
print(parallel.most_common(5))

if name == "main":
    main()

```

Pathway B: Parallel Image Metadata

This is good for students working on image classification.

```

from pathlib import Path

from concurrent.futures import ProcessPoolExecutor

from PIL import Image, ImageStat

import time

```

```

IMAGE_DIR = Path("images")

```

```

def load_image_metadata(path):
    """
    CPU-ish image metadata extraction.

```

This may include image decoding and basic statistics.

```
"""
```

```
with Image.open(path) as img:
```

```
    img = img.convert("RGB")
```

```
    width, height = img.size
```

```
    stat = ImageStat.Stat(img)
```

```
    brightness = sum(stat.mean) / 3.0
```

```
return {
```

```
    "path": str(path),
```

```
    "width": width,
```

```
    "height": height,
```

```
    "brightness": brightness,
```

```
}
```

```
def serial_image_metadata(paths):
```

```
    results = []
```

```
    errors = []
```

```
    for path in paths:
```

```
        try:
```

```
            results.append(load_image_metadata(path))
```

```
        except Exception as e:
```

```
            errors.append((str(path), str(e)))
```

```
return results, errors
```

```
def parallel_image_metadata(paths, workers=4, chunksize=10):
```

```
    results = []
```

```
    errors = []
```

```
    with ProcessPoolExecutor(max_workers=workers) as executor:
```

```
        futures = {
```

```
            executor.submit(load_image_metadata, path): path
```

```
            for path in paths
```

```
        }
```

```
        for future, path in futures.items():
```

```
            try:
```

```
                results.append(future.result())
```

```
            except Exception as e:
```

```
                errors.append((str(path), str(e)))
```

```
    return results, errors
```

```
def summarize_images(records):
```

```
    if not records:
```

```
        return {}
```

```
    return {
```

```
        "count": len(records),
```

```
"avg_width": sum(r["width"] for r in records) / len(records),  
"avg_height": sum(r["height"] for r in records) / len(records),  
"avg_brightness": sum(r["brightness"] for r in records) / len(records),  
}
```

```
def main():
```

```
    paths = (  
        list(IMAGE_DIR.glob(".jpg"))  
        + list(IMAGE_DIR.glob(".jpeg"))  
        + list(IMAGE_DIR.glob("*.png"))  
    )
```

```
    print("number of images:", len(paths))
```

```
    start = time.perf_counter()
```

```
    serial_results, serial_errors = serial_image_metadata(paths)
```

```
    serial_time = time.perf_counter() - start
```

```
    start = time.perf_counter()
```

```
    parallel_results, parallel_errors = parallel_image_metadata(  
        paths,
```

```
        workers=4,
```

```
    )
```

```
    parallel_time = time.perf_counter() - start
```

```
    print("serial time:", serial_time)
```



```
print("parallel time:", parallel_time)

print("speedup:", serial_time / parallel_time if parallel_time else None)

print("serial summary:", summarize_images(serial_results))

print("parallel summary:", summarize_images(parallel_results))
```

```
if name == "main":
```

```
    main()
```

Note: for image loading, performance depends heavily on disk speed, image size, decoder behavior, and whether the work is truly CPU-bound or partly I/O-bound. If students observe that threads beat processes for image loading, that is a good discussion point.

Pathway C: Parallel Tabular Chunk Statistics

This is good for students working on data analysis.

```
import random
```

```
import time
```

```
from concurrent.futures import ProcessPoolExecutor
```

```
def make_rows(n=200_000):
```

```
    rows = []
```

```
    for i in range(n):
```

```
        rows.append({
```

```
            "x": random.random(),
```

```
            "y": random.random(),
```

```
            "label": random.choice(["A", "B", "C"]),
```

```
        })
```

```
return rows
```

```
def chunk_list(items, chunk_size):  
    for i in range(0, len(items), chunk_size):  
        yield items[i:i + chunk_size]
```

```
def process_rows(rows):  
    total_x = 0.0  
    total_y = 0.0  
    count = 0  
    label_counts = {}  
  
    for row in rows:  
        # Simulate CPU-heavy row transformation  
        x = row["x"]  
        y = row["y"]  
  
        for _ in range(100):  
            x = (x * 1.0001 + y * 0.9999) % 1.0  
  
        total_x += x  
        total_y += y  
        count += 1  
  
        label = row["label"]  
        label_counts[label] = label_counts.get(label, 0) + 1
```

```
return {  
    "total_x": total_x,  
    "total_y": total_y,  
    "count": count,  
    "label_counts": label_counts,  
}
```

```
def reduce_stats(partials):  
    total_x = sum(p["total_x"] for p in partials)  
    total_y = sum(p["total_y"] for p in partials)  
    count = sum(p["count"] for p in partials)  
  
    label_counts = {}  
  
    for p in partials:  
        for label, value in p["label_counts"].items():  
            label_counts[label] = label_counts.get(label, 0) + value  
  
    return {  
        "mean_x": total_x / count,  
        "mean_y": total_y / count,  
        "count": count,  
        "label_counts": label_counts,  
    }
```

```

def serial_stats(rows):
    return reduce_stats([process_rows(rows)])

def parallel_stats(rows, workers=4, chunk_size=10_000):
    chunks = list(chunk_list(rows, chunk_size))

    with ProcessPoolExecutor(max_workers=workers) as executor:
        partials = list(executor.map(process_rows, chunks))

    return reduce_stats(partials)

def main():
    rows = make_rows()

    start = time.perf_counter()
    serial = serial_stats(rows)
    serial_time = time.perf_counter() - start

    start = time.perf_counter()
    parallel = parallel_stats(rows, workers=4, chunk_size=10_000)
    parallel_time = time.perf_counter() - start

    print("serial time:", serial_time)
    print("parallel time:", parallel_time)
    print("speedup:", serial_time / parallel_time)
    print("serial:", serial)

```

```
print("parallel:", parallel)
```

```
if name == "main":
```

```
    main()
```

Stretch Experiment 1: Task Size

Ask students to change:

```
CPU_WORK = 5_000
```

```
CPU_WORK = 30_000
```

```
CPU_WORK = 100_000
```

Stretch Experiment 2: Worker Count

Have students test:

```
WORKER_COUNTS = [1, 2, 4, 8, 16]
```

Questions

1. Does speedup keep increasing?
 2. Does the best worker count match `os.cpu_count()`?
 3. What happens if workers exceed physical cores?
 4. Did memory usage increase?
-

Stretch Experiment 3: Chunk Size

Have students test:

```
CHUNK_SIZES = [1, 10, 100, 1000, 5000, 10000]
```

Questions

1. Why is `chunk_size=1` often inefficient?
2. Why can very large chunks reduce load balance?

3. Which chunk size was best?
4. Does best chunk size change when CPU_WORK changes?

Deliverables

Send the following to me by email, lidongvtemail@gmail.com

1. Results Table

Must include:

- method
- workers
- chunk size
- runtime
- speedup
- efficiency
- items/sec
- correctness

2. One Chart

Recommended:

x-axis: workers

y-axis: speedup

Second chart:

x-axis: chunk size

y-axis: runtime

4. Short Reflection

Suggested prompt:

In 1–2 paragraphs:

1. Which method was fastest?
 2. Which worker count and chunk size worked best?
 3. Why did speedup flatten?
 4. What overheads did you observe?
 5. How could map-reduce help your final project?
-